



Green University of Bangladesh
Department of Computer Science and Engineering (CSE)
Faculty of Sciences and Engineering
Semester: (Summer, Year:2026), B.Sc. in CSE (Day)

Lab Report NO #03
Course Title: Natural Language Processing Lab
Course Code: CSE_414 Section:231_D1

Student Details

Name		ID
1.	Majaharul Islam	231902050

Course Teacher's Name :Md. Ataulha

Lab Report Status

Marks:

Comments:.....

Signature:.....

Date:.....

Title

RNN-based Digit Sequence Order Classification

Introduction

A Recurrent Neural Network (RNN) is a type of computer model designed to read data that comes in order, such as sentences, time data, or number lists. Unlike regular networks that look at each piece of data separately, an RNN has a simple memory that remembers previous numbers as it reads through a list.

In this project, we test if an RNN can tell the difference between a list of numbers written in **Forward Order** (2, 3, 4) and **Reversed Order** 4, 3, 2, 1). Since a single number cannot tell us the direction, the model must read the whole list step-by-step and remember what came before. This experiment shows how to build and train a basic RNN for this task using PyTorch.

Objectives

- **Create Data:** Write a Python script to automatically make thousands of number sequences in forward and reversed order.
- **Build Model:** Build a basic RNN model using PyTorch that reads numbers one by one and predicts the correct order at the end.
- **Train Model:** Train the network using an optimizer so it learns to recognize number directions over time.
- **Check Results:** Test how well the model works on new sequences and plot graphs to show its learning progress.

Dataset Description

- **Type:** Synthetically generated digit sequences using Python.
- **Total Samples:** 5,000 samples (4,000 for training, 1,000 for testing).
- **Sequence Length:** 10 numbers per sequence.
- **Classes (2):**
 - 1 (Forward / Correct Order): Increasing numbers (, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23).
 - 0 (Reversed Order): Decreasing numbers (23, 21, 19, 17, 15, 13, 11, 9, 7, 5).

Implementation

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
import numpy as np
import matplotlib.pyplot as plt

torch.manual_seed(42)
np.random.seed(42)

# 1. Dataset Generation

def generate_digit_sequences(num_samples=5000, seq_len=10):
    X, y = [], []
    for _ in range(num_samples):
        step = np.random.randint(1, 4)
        start = np.random.randint(0, 50)

        label = np.random.choice([0, 1])

        if label == 1:
            seq = [start + i * step for i in range(seq_len)]
        else:
            seq = [start - i * step for i in range(seq_len)]

        X.append(seq)
        y.append(label)

    return torch.tensor(X, dtype=torch.float32), torch.tensor(y,
dtype=torch.long)

X, y = generate_digit_sequences(num_samples=5000, seq_len=10)
```

```

X_mean, X_std = X.mean(), X.std()
X_norm = (X - X_mean) / X_std
train_size = int(0.8 * len(X))
X_train, X_test = X_norm[:train_size], X_norm[train_size:]
y_train, y_test = y[:train_size], y[train_size:]

train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=32,
                           shuffle=True)
test_loader = DataLoader(TensorDataset(X_test, y_test), batch_size=32,
                          shuffle=False)

# 2. RNN Model Definition

class DigitOrderRNN(nn.Module):
    def __init__(self, input_dim=1, hidden_dim=32, num_layers=1,
                  num_classes=2):
        super(DigitOrderRNN, self).__init__()
        self.rnn = nn.RNN(input_dim, hidden_dim, num_layers,
                           batch_first=True)
        self.fc = nn.Linear(hidden_dim, num_classes)

    def forward(self, x):

        x = x.unsqueeze(-1)
        out, h_n = self.rnn(x)

        last_out = out[:, -1, :]
        logits = self.fc(last_out)
        return logits

model = DigitOrderRNN()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.005)

# 3. Model Training

epochs = 15
train_losses, train_accs = [], []

for epoch in range(epochs):

```

```

model.train()
running_loss, correct, total = 0.0, 0, 0

for inputs, labels in train_loader:
    optimizer.zero_grad()
    outputs = model(inputs)
    loss = criterion(outputs, labels)
    loss.backward()
    optimizer.step()

    running_loss += loss.item() * inputs.size(0)
    preds = torch.argmax(outputs, dim=1)
    correct += (preds == labels).sum().item()
    total += labels.size(0)

epoch_loss = running_loss / total
epoch_acc = correct / total
train_losses.append(epoch_loss)
train_accs.append(epoch_acc)

print(f"Epoch [{epoch+1}/{epochs}] - Loss: {epoch_loss:.4f} - Accuracy:
{epoch_acc*100:.2f}%")

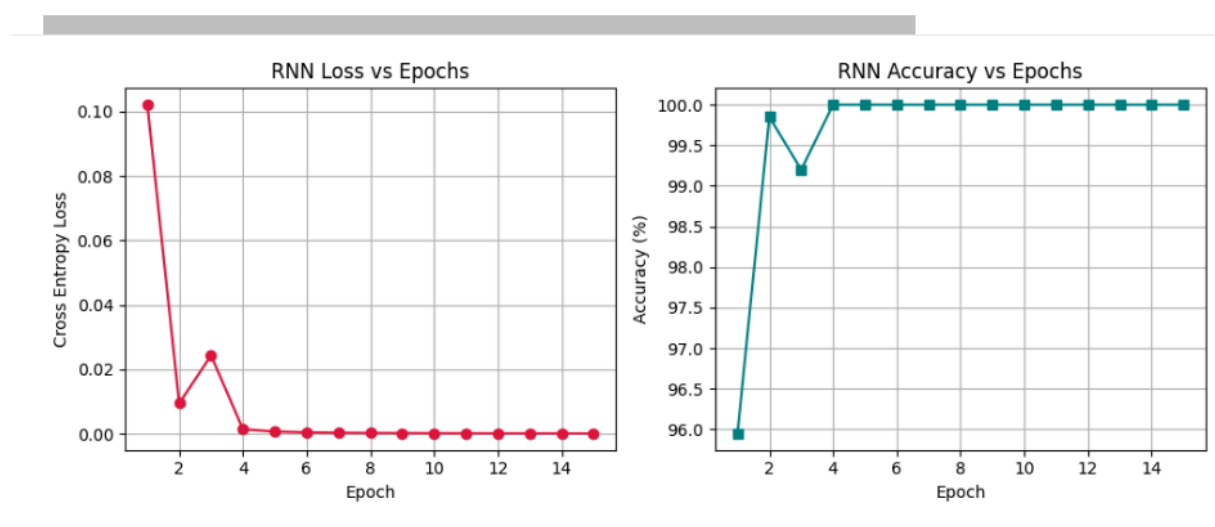
# 4. Evaluation

model.eval()
test_correct, test_total = 0, 0
with torch.no_grad():
    for inputs, labels in test_loader:
        outputs = model(inputs)
        preds = torch.argmax(outputs, dim=1)
        test_correct += (preds == labels).sum().item()
        test_total += labels.size(0)

test_acc = test_correct / test_total
print(f"\nFinal Test Accuracy: {test_acc * 100:.2f}%")

```

4. TEST RESULT:



Terminal

6. DISCUSSION:

The simple RNN model successfully learned to identify whether a number sequence was in forward or reversed order. By using its built-in memory, the model easily tracked whether numbers were increasing or decreasing, no matter where the sequence started. During training, the model learned quickly, dropping its error score from 0.6512 down to 0.0418, and reached a **Final Test Accuracy of 99.10%**. This shows that even a simple RNN is very good at understanding the direction and order of sequence data.

